

DSP LAB

EXP 3: Linear convolution

Toeplitz

```
import numpy as np
#input the input sequence x[n] and impulse response h[n]
x = np.array(input("Enter the elements of the input sequence :").split(",")).astype(int)
h = np.array(input("Enter the elements of the impulse response :").split(",")).astype(int)
hn = h
row = len(x)
column = len(x)+len(h)-1
if len(h)<column:
    z = np.zeros(column-len(h))
    h = np.concatenate((h,z))
#toeplitz matrix
toeplitz = h
for i in range (row-1):
    h = np.roll(h,1) #to circular shift elements of array
    toeplitz = np.concatenate((toeplitz,h))
toeplitz.shape = (row,column)
toeplitz = np.transpose(toeplitz)
print("Toeplitz Matrix :\n",toeplitz)
print("The output sequence (using toeplitz matrix) is :",np.dot(toeplitz,x).astype(int))
#verifying the output
print("The output sequence (using numpy.convolve) is :", np.convolve(x,hn))
```

mathematical expression

```
x = [int(i) for i in input("Enter elements of the input sequence : ").split(',')]
h = [int(i) for i in input("Enter elements of the impulse response : ").split(',')]

# Length of the linear convolution result
N = len(x) + len(h) - 1

# Initialize the result list with zeros
y = [0] * N

# Compute the linear convolution using the direct mathematical expression
for n in range(N):
    for k in range(len(h)):
        if n - k >= 0 and n - k < len(x):
            y[n] += x[n - k] * h[k]
```

```
# Print the result
print("Linear Convolution using Mathematical expression : ", y)
```

EXP 4: DFT

```
import numpy as np
import matplotlib.pyplot as plt
import cmath as cm
x=np.array(input("enter the sequence:").split(",")).astype(complex)
N=len(x)
X=[]
X1=np.zeros(N)
j=cm.sqrt(-1)
w=[]
for k in range(N):
    sum=0
    a=0
    for n in range(N):
        a=x[n]*np.exp((-j)*2*np.pi*k*n/N)
        sum+=a
        sum=np.around(sum,0)
        X.append(sum)
    k=0
    n=0
for k in range(N):
    a=0
    for n in range(N):
        a=cm.exp((-j)*2*np.pi*k*n/N)
        a=np.around(a,0)
        w.append(a)
w=np.array(w).reshape(N,N)
print("twiddle factor")
print(w)
X1=x.dot(w)
print("DFT math:",X)
print("DFT MATRIX:",X1)
def graph(x):
    m=[]
    p=[]
    for i in range(N):
        m.append(abs(x[i]))
        p.append(cm.phase(x[i]))
```

```

print("magnitude:",np.around(m,1))
print("phase:",np.around(p,1))
fig,a=plt.subplots(2,1)
a[0].stem(m)
a[0].set_xlabel("time")
a[0].set_ylabel("magnitude")
a[0].set_title("magnitude response")
a[1].stem(p)
a[1].set_xlabel("time")
a[1].set_ylabel("phase")
a[1].set_title("phase response")
fig.tight_layout()
plt.show()
graph(X)
graph(X1)

```

PT DFT is sampled DTFT

```

import numpy as np
import matplotlib.pyplot as plt
import cmath as cm

# Input sequence from user
x = [int(i) for i in input("Enter elements of input sequence: ").split(',')]
N = len(x) # Length of the sequence

# Initialize the DFT array
dft = []
j = cm.sqrt(-1)

# Compute the DFT using the summation formula
for k in range(N):
    s = 0 # Reset the sum for each k
    for n in range(N):
        s += x[n] * np.exp((-1j) * 2 * np.pi * n * k / N) # DFT formula
    p = complex(round(s.real, 4), round(s.imag, 4)) # Round the real and imaginary parts
    dft.append(p)

print("DFT result =", dft)

# Frequency range for DFT (normalized from 0 to 2*pi)
F = np.linspace(0, 2 * np.pi, N, endpoint=False)

```

```

# Compute DTFT for a finer frequency range
W = np.linspace(0, 2 * np.pi, 1000) # Fine frequency range for DTFT
DTFT = np.array([np.sum(x * np.exp(complex(0, -1) * w * np.arange(N))) for w in W])

# Plot DTFT and DFT
plt.figure(figsize=(12, 6))
plt.plot(W, np.abs(DTFT), label='DTFT', color='b') # DTFT plot (continuous)
plt.stem(F, np.abs(dft), 'r', markerfmt='ro', baselfmt="", label='DFT') # DFT plot (discrete)
plt.xlabel('Frequency (radians)')
plt.ylabel('Magnitude')
plt.title("DTFT and DFT of the input sequence")
plt.legend()
plt.grid()
plt.show()

```

EXP 5: DFT properties

```

import numpy as np

def circular_convolution(x,h):
    N=max(len(x),len(h))
    if len(x)<N:
        x=x+[0]*(N-len(x))
    if len(h)<N:
        h=h+[0]*(N-len(h))
    y=np.zeros(N,dtype=complex)
    for n in range(N):
        for k in range(N):
            y[n]+=x[k]*h[(n-k)%N]
    return y

# def dft(x):
#     return np.fft.fft(x)

#Defining Samples
x1=np.array([1,2,3,4])
x2=np.array([4,3,2,1])
print("x1(n)=",x1)
print("x2(n)=",x2)
print()
X1=np.fft.fft(x1)
X2=np.fft.fft(x2)
N=len(x1)

```

#Linearity

```
a=int(input("enter a:"))
b=int(input("enter b:"))
print()

# for n in range (N):
lhs=np.fft.fft(a*x1+b*x2)
rhs=a*X1+b*X2
print("linearity:")
print("a*x1(n)+b*x2(n)=",lhs)
print("a*X1(k)+b*X2(k)=",rhs)
if np.allclose(lhs,rhs):
    print("linearity property is verified")
else:
    print("linearity property is not verified")
print()
```

#Circular Shift

```
k=1
x_shifted=np.roll(x1,k)
X_shifted=np.fft.fft(x_shifted)
X1_shifted=X1*np.exp(-1j*2*np.pi*k*np.arange(N)/N)
X1_shifted=np.around(X1_shifted,4)
print("circular shift:")
print("Y[K]=",X_shifted)
print("X[K]*e^(-j*(2*np.pi*N)*k*n)=",X1_shifted)
if np.allclose(X_shifted,X1_shifted):
    print("circular shift property is verified!")
else:
    print("circular shift property is not verified")
print()
```

#Duality

```
X1_DFT=np.fft.fft(X1)
nx=[N*x1[-n] for n in range(N)]
Nx=[complex(j) for j in nx]
print("duality theorem:")
print("DFT(DFT(x[n])):",X1_DFT)
print("N*x(-n):",Nx)
if np.allclose(X1_DFT,Nx):
```

```

    print("duality property is verified")
else:
    print("duality property is not verified")
print()

#Multiplication(Convolution Theorem)
x_product=np.multiply(x1,x2)
X_product=np.fft.fft(x_product)
X1_conv_X2= circular_convolution(X1,X2)/N
print("Multiplication(Convolution Theorem):")
print("DFT(x1(n)*x2(n))=", X_product)
print("(1/N)(X[K]*Y[K])=", X1_conv_X2)
if np.allclose(X_product,X1_conv_X2):
    print("multiplication(convolution theorem) property is verified")
else:
    print("multiplication(convolution theorem) property is not verified")
print()

```

```

#Circular Convolution
circular_conv=circular_convolution(x1,x2)
print("Circular Convolution:")
print("Circular Convolution of x1 and x2:", circular_conv)
print("IDFT(DFT(x1)*(DFT(x2)):",np.fft.ifft(X1*X2))
if np.allclose(circular_conv,np.fft.ifft(X1*X2)):
    print("Circular Convolution property is verified")
else:
    print("Circular Convolution property is not verified")
print()

```

```

#Parseval's Theorem
energy_time=np.sum(np.abs(x1)**2)
energy_freq=np.sum(abs(X1)**2)/N
print("Parseval's Theorem:")
print("Sum of |x1[n]|^2=",energy_time)
print("Sum of |X1[K]|^2=",energy_freq)
if np.allclose(energy_time,energy_freq):
    print("Parseval's Theorem is verified")
else:
    print("Parseval's Theorem is not verified")
print()

```

```

EXP 6
import numpy as np
def dft(x):
    N=len(x)

```

```

X=np.zeros(N,dtype=complex)
for k in range(N):
    for n in range (N):
        X[k]+=x[n]*np.exp(-1j*2*np.pi*k*n/N)
return X

def idft(X):
    N=len(X)
    x=np.zeros(N,dtype=complex)
    for n in range(N):
        for k in range(N):
            x[n]+=X[k]*np.exp(1j*2*np.pi*k*n/N)
    return x/N

def linear_convolution_using_circular(x,h):
    L=len(x)
    M=len(h)
    N=L+ M-1
    x_padded=x + [0]*(N-L)
    h_padded=h+ [0]*(N-M)
    X= dft(x_padded)
    H=dft(h_padded)
    Y=X*H
    y1=idft(Y)
    y_linear=np.real(y1)
    return y_linear

x=[int(i) for i in input ("enter elements of input sequence:".split(','))]
h=[int(i) for i in input ("eneter elements of inputs response:".split(','))]

y=linear_convolution_using_circular(x,h)
print("linear convolution result:",y)

```

EXP 6: Linear Convolution using Circular

```

import numpy as np
def dft(x):
    N=len(x)
    X=np.zeros(N,dtype=complex)
    for k in range(N):
        for n in range (N):
            X[k]+=x[n]*np.exp(-1j*2*np.pi*k*n/N)
    return X

```

```

def idft(X):
    N=len(X)
    31
    x=np.zeros(N,dtype=complex)
    for n in range(N):
        for k in range(N):
            x[n]+=X[k]*np.exp(1j*2*np.pi*k*n/N)
    return x/N
def linear_convolution_using_circular(x,h):
    L=len(x)
    M=len(h)
    N=L+ M-1
    x_padded=x + [0]*(N-L)
    h_padded=h+ [0]*(N-M)
    X= dft(x_padded)
    H=dft(h_padded)
    Y=X*H
    y1=idft(Y)
    y_linear=np.real(y1)
    return y_linear
x=[int(i) for i in input ("enter elements of input sequence:").split(',')]
h=[int(i) for i in input ("enter elements of inputs response:").split(',')]
y=linear_convolution_using_circular(x,h)
print("linear convolution result:",y)

```

DIT FFT

```

import numpy as np
a=np.array(input("enter the input:").split(",")).astype(int)
def DIT_FFT(x):
    n=len(x)
    if n<=1:
        return(x)
    even=DIT_FFT(x[0::2])
    odd=DIT_FFT(x[1::2])
    c=[0]*n
    for k in range(n//2):
        t=np.exp(-2j*np.pi*k/n)*odd[k]
        c[k]=even[k]+t
        c[k+n//2]=even[k]-t
    return(c)

```



```
print("result",DIT_FFT(a))
```

FIR FILTER DESIGN LPF

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.signal as sp
```

```
# Define the Hanning window function
```

```
def hanning(n, N):
    x = 0.5 + 0.5 * np.cos(2 * np.pi * n / (N - 1))
    return x
```

```
# Define the Hamming window function
```

```
def hamming(n, N):
    x = 0.54 + 0.46 * np.cos(2 * np.pi * n / (N - 1))
    return x
```

```
# Function to plot magnitude and phase response
```

```
def mfreq(b):
    w, h = sp.freqz(b, 1)
    db = 20 * np.log10(abs(h)) # Corrected log10
    hphase = np.unwrap(np.angle(h))
```

```
fig, a = plt.subplots(2, 1, figsize=(10, 6))
a[0].plot(w, db)
a[0].set_title('Magnitude Response')
a[0].set_xlabel('Frequency [rad/sample]')
a[0].set_ylabel('Magnitude [dB]')
a[0].grid()
```

```
a[1].plot(w, hphase)
a[1].set_title('Phase Response')
a[1].set_xlabel('Frequency [rad/sample]')
a[1].set_ylabel('Phase [radians]')
a[1].grid()
```

```
plt.tight_layout()
plt.show()
```

```
# Function to create a low-pass filter with specified window
```

```
def lpf(N, wc, win):
    a = (N - 1) / 2
```

```

h = [wc / np.pi if i == a else np.sin(wc * (i - a)) / (np.pi * (i - a)) for i in np.arange(0, N)]

# Apply the selected window function
windows = [hanning, hamming]
window_values = windows[win - 1](np.arange(0, N), N)
h_ = h * window_values

# Plot the frequency response of the filtered coefficients
mfreq(h_)

# User input for filter parameters
N = int(input("Enter N: "))
wc = float(input("Enter wc (cutoff frequency in radians): "))
win = int(input("Choose window\n1. Hanning\n2. Hamming\n"))

# Call the low-pass filter function with provided inputs
lpf(N, wc, win)

```

Block Convolution

Overlap Save

import numpy as np

```

def overlap_save(x, h, L):
    M = len(h)
    N = L + M - 1
    h = np.concatenate((h, np.zeros(N - M))) # Zero-pad impulse response to length N
    H = np.fft.fft(h, N)

    # Zero-pad the input sequence at the beginning with M - 1 zeros and at the end with M - 1
    zeros
    x = np.concatenate((np.zeros(M - 1), x, np.zeros(M - 1)))
    y = np.zeros(len(x) - M + 1) # Output array

    # Process each block of the input sequence
    for i in range(0, len(x) - N + 1, L):
        x_block = x[i : i + N] # Take a block of length N from the input
        X_block = np.fft.fft(x_block, N) # FFT of the block
        y_block = np.fft.ifft(X_block * H).real # Inverse FFT of product gives block convolution
    result

```

```

        # Discard the first M-1 samples and keep the rest
        y[i : i + L] = y_block[M - 1:] # Retain only the valid section

    return y

# Input and function call
x = [int(i) for i in input("Enter elements of the input sequence: ").split(',')]
h = [int(i) for i in input("Enter elements of the impulse response: ").split(',')]
L = len(x) + len(h) - 1
y_ols = overlap_save(x, h, L)

print("Overlap-Save Convolution:", y_ols)

```

Overlap ADD

```

import numpy as np
def overlap_add(x, h, L):
    M = len(h)
    N = L + M - 1
    h = np.concatenate((h, np.zeros(N - M))) # Zero-pad impulse response
    H = np.fft.fft(h, N)
    y = np.zeros(len(x) + M - 1)

    # Process blocks of the input sequence
    for i in range(0, len(x), L):
        x_block = x[i:i + L] # Take a block of length L from the input
        if len(x_block) < L:
            x_block = np.concatenate((x_block, np.zeros(L - len(x_block)))) # Zero-pad the last
            block if necessary

        x_block = np.concatenate((x_block, np.zeros(N - L))) # Zero-pad the block to length N
        X_block = np.fft.fft(x_block, N) # FFT of the block
        y_block = np.fft.ifft(X_block * H).real # Inverse FFT of product gives the block convolution
    result = y

    start_idx = i
    end_idx = min(start_idx + N, len(y))
    y[start_idx:end_idx] += y_block[:end_idx - start_idx] # Add the block result to the output
    array

    result = [int(i) for i in y]

```

```
return result
```

```
# User input and function call
```

```
x = [int(i) for i in input("Enter elements of the input sequence: ").split(', ')]
```

```
h = [int(i) for i in input("Enter elements of the impulse response: ").split(', ')]
```

```
L = int(input("Enter the block length:"))
```

```
y_ola = overlap_add(x, h, L)
```

```
print("Overlap-Add Convolution:", y_ola)
```

DSK6713

Linear Convolution

```
#include<stdio.h>
```

```
void conv(float *x, float *h, float *y, short l, short m);
```

```
void main()
```

```
{
```

```
floatx[] = { 1, 2, 3, 4 };
```

```
floath[] = { 1, 1, 1 };
```

```
floaty[6]; //length of y = L+M-1
```

```
short n;
```

```
conv(x, h, y, 4, 3);
```

```
for (n = 0; n < 6; n++)
```

```
printf("%.2f ", y[n]);
```

```
}
```

```
voidconv(float *x, float *h, float *y, short l, short m)
```

```
{
```

```
short k, kmin, kmax, n;
```

```
for (n = 0; n < l + m - 1; n++)
```

```
{
```

```
y[n] = 0;
```

```
86
```

```
kmin = (n > l - 1) ? n - l + 1 : 0;
```

```
kmax = (n > m - 1) ? m - 1 : n;
```

```
//printf("%d %d\n", kmin, kmax);
```

```
for (k = kmin; k <= kmax; k++)
```

```
{
```

```
y[n] = y[n] + h[k] * x[n - k];
```

```
}  
}  
}
```

Ring Counter

```
#include <dsk6713.h>  
#include <dsk6713_led.h>  
#include <dsk6713_dip.h>  
void main()  
{  
while(1)  
{  
    if (DSK6713_DIP_get(0) == 0)  
    {  
        for(i=0; i<4; i++)  
        {  
            DSK6713_LED_on(i);  
            DSK6713_waitusec(200000);  
        }  
        for(i=0; i<4; i++)  
        {  
            DSK6713_LED_off(i);  
            DSK6713_waitusec(200000);  
        }  
    }  
    else  
    {  
        for(i=0; i<4; i++)  
            DSK6713_LED_off(i);  
    }  
}  
}
```

Johnson Counter

```
#include <dsk6713.h>  
#include <dsk6713_led.h>  
#include <dsk6713_dip.h>  
void main()  
{
```

```

while(1)
{
    if (DSK6713_DIP_get(0) == 0)
    {
        // Code to toggle LEDs on and off sequentially
        for(i=0; i<4; i++)
        {
            DSK6713_LED_on(i);
            DSK6713_waitusec(200000);
        }
        for(i=0; i<4; i++)
        {
            DSK6713_LED_off(i);
            DSK6713_waitusec(200000);
        }
        for(i=0; i<4; i++)
        {
            DSK6713_LED_on(i);
            DSK6713_waitusec(200000);
            DSK6713_LED_off(i);
        }

        }
        else
        {
            // Code to blink LEDs one by one
            for(i=0; i<4; i++)
            {
                DSK6713_LED_on(i);
                DSK6713_waitusec(200000);
                DSK6713_LED_off(i);
            }

        }
    }
}

```

LAB 1 Qs

8 point DFT using matrix method

import numpy as np

import matplotlib.pyplot as plt

```

import cmath as cm

x=np.array(input("enter the sequence:").split(",")).astype(complex)
N=8

X1=np.zeros(N)
j=cm.sqrt(-1)
w=[]

if len(x)<N:
    z = np.zeros(N-len(x))
    x = np.concatenate((x,z))

k=0
n=0
for k in range(N):
    a=0
    for n in range(N):
        a=cm.exp((-j)*2*np.pi*k*n/N)
        # a=np.around(a,0)
        w.append(a)
w=np.array(w).reshape(N,N)
print("twiddle factor")
print(w)
X1=x.dot(w)

print("DFT MATRIX:",X1)

```

DFT Calculation

```
import numpy as np
import matplotlib.pyplot as plt
import cmath as cm

x=np.array(input("enter the sequence:").split(",")).astype(complex)
N=4
X=[]
X1=np.zeros(N)
j=cm.sqrt(-1)
w=[]

#zero padding
if len(x)<N:
    z=np.zeros(N-len(x))
    x=np.concatenate((x,z))

#using eqn
for k in range(N):
    sum=0
    a=0
    for n in range(N):
        a=x[n]*np.exp((-j)*2*np.pi*k*n/N)
        sum+=a
    sum=np.around(sum,0)
    X.append(sum)

# twiddle factor
k=0
n=0
for k in range(N):
    a=0
    for n in range(N):
        a=cm.exp((-j)*2*np.pi*k*n/N)
        a=np.around(a,0)
        w.append(a)
w=np.array(w).reshape(N,N)
print("twiddle factor")
print(w)
X1=x.dot(w)

print("DFT math:",X)
print("DFT MATRIX:",X1)
```



```

#magnitude and phase plotting
def graph(x):
    m=[]
    p=[]
    for i in range(N):
        m.append(abs(x[i]))
        p.append(cm.phase(x[i]))
    print("magnitude:",np.around(m,1))
    print("phase:",np.around(p,1))

fig,a=plt.subplots(2,1)
a[0].stem(m)
a[0].set_xlabel("time")
a[0].set_ylabel("magnitude")
a[0].set_title("magnitude response")
a[1].stem(p)
a[1].set_xlabel("time")
a[1].set_ylabel("phase")
a[1].set_title("phase response")
fig.tight_layout()
plt.show()
graph(X)
graph(X1)

```

CIRCULAR CONVOLUTION

Toplitz

```
import numpy as np
```

```
#input the input sequence x[n] and impulse response h[n]
```

```
x = np.array(input("Enter the elements of the input sequence :").split(",")).astype(int)
```

```
h = np.array(input("Enter the elements of the impulse response :").split(",")).astype(int)
```

```
hn = h
```

```
M=len(h)
```

```
L = len(x)
```

```
N= len(x)+len(h)-1
```

```
if M<N:
```

```
    z = np.zeros(N-M))
```

```
    h = np.concatenate((h,z))
```

```
#toplitz matrix
```

```

toplitz = h
for i in range (L-1):
    h = np.roll(h,1) #to circular shift elements of array
    toplitz = np.concatenate((toplitz,h))
toplitz.shape = (L,N)
toplitz = np.transpose(toplitz)

print("Toeplitz Matrix :\n",toplitz)
print("The output sequence (using toeplitz matrix) is :",np.dot(toplitz,x).astype(int))

#verifying the output
print("The output sequence (using numpy.convolve) is : ", np.convolve(x,hn))

```

```

def circular_convolution(x,h):
    N=max(len(x),len(h))
    if len(x)<N:
        x=x+[0]*(N-len(x))
    if len(h)<N:
        h=h+[0]*(N-len(h))
    y=np.zeros(N,dtype=complex)
    for n in range(N):
        for k in range(N):
            y[n]+=x[k]*h[(n-k)%N]
    return y

circular_conv=circular_convolution(x1,x2)
print("Circular Convolution:")
print("Circular Convolution of x1 and x2:", circular_conv)
print("IDFT(DFT(x1)*(DFT(x2)):",np.fft.ifft(X1*X2))
if np.allclose(circular_conv,np.fft.ifft(X1*X2)):
    print("Circular Convolution property is verified")
else:
    print("Circular Convolution property is not verified")
print()

```

```

#include <math.h>
#include <dsk6713.h>
#include <dsk6713_aic23.h>
int main()
{
    float Fs = 8000.;
    float F0 = 1000.;
    float pi = 3.141592653589;
    float theta = 0.;
    float delta = 2. * pi * F0 / Fs; // increment for theta
    float sample;
    unsigned out_sample;
    /* Initialize the board support library, must be called first */
    DSK6713_init();

    DSK6713_AIC23_Config config = DSK6713_AIC23_DEFAULTCONFIG;
    DSK6713_AIC23_CodecHandle hCodec;
    /* Start the codec */
    hCodec = DSK6713_AIC23_openCodec(0, &config);

    /* Change the sampling rate to 16 kHz */
    DSK6713_AIC23_setFreq(hCodec, DSK6713_AIC23_FREQ_8KHZ);

    for (;;)
    { /* Infinite loop */
        sample = 15000.0 * sin(theta); /* Scale for DAC */
        out_sample = (int)sample & 0x0000ffff; // Put in lower half (R)

        /* Poll XRDY bit until true, then write to DXR */
        while (!DSK6713_AIC23_write(hCodec, out_sample)) ;
        theta += delta;
    }
}

```

https://docs.google.com/document/d/1viG8hAhK1lIBCo_mRmYhrmN6bvZp7VivwH0jlmC1f4/edit?usp=drivesdk

```

import numpy as np
# Function to compute the DIFFFT in-place
def diffft_in_place(x):

```

```

N = len(x)
# print(N)
stages = int(np.log2(N)) # The number of stages (log2(N))
# print(stages)
# Perform the DIFFFT in place
for stage in range(stages):
    num_groups = 2 ** stage # Number of butterfly groups
    group_size = N // num_groups # Size of each butterfly group
    print(stage,num_groups,group_size)
    # For each butterfly group
    for group in range(num_groups):
        for k in range(group_size // 2):
            print(group,k)
            even_index = group * group_size + k
            odd_index = even_index + group_size // 2
            twiddle_factor = np.exp(-2j * np.pi * k / group_size) #  $W_N^k$ 
            print(even_index,odd_index,twiddle_factor)
            # Compute the butterfly for current pair
            t1 = x[even_index]
            t2 = x[odd_index]
            x[even_index] = t1 + t2
            x[odd_index] = t1*twiddle_factor - t2*twiddle_factor

        print(x)

    # Show intermediate result after each stage
    print(f"Stage {stage + 1}: {x}")
return x

```

Q1. 8 point and 16 point dft using matrix method.

```
import numpy as np
import cmath as cm

x = np.array(input("Enter x(n): ").split(',')).astype(complex)

N = 8 # 8-point DFT
N1 = 16 # 16-point DFT

def twiddlefactor(N):
    w=[]
    j=cm.sqrt(-1)
    k=0
    n=0
    for k in range (N):
        a=0
        for n in range(N):
            a=np.exp(((j)*np.pi*k*n)/N)
            a=np.around(a,0) # this can be commented for checking with the built in fn
            w.append(a)
        w=np.array(w).reshape(N,N)
    return w

w8 = twiddlefactor(N) # 8-point DFT matrix
w16 = twiddlefactor(N1) # 16-point DFT matrix

# Zero-pad the input for 4-point and 8-point DFTs
x_padded_8 = np.pad(x, (0, N - len(x)))
x_padded_16 = np.pad(x, (0, N1 - len(x)))

# Calculate the DFTs using matrix multiplication
X8 = x_padded_8.dot(w8)
X16 = x_padded_16.dot(w16)

print("4-point DFT result:\n", X8)
print("\n 8-point DFT result:\n", X16)
print("\n \n")
# print("8-point DFT result:\n", np.fft.fft(x_padded_8))
# print("\n 16-point DFT result:\n", np.fft.fft(x_padded_16))
```

Q2. Find the impulse response using twiddle factor matrix method

```
import numpy as np

x = np.array(input("Enter x(n): ").split(',')).astype(complex)
N = len(x)

def twiddlefactor(N):
    w=[]
    k=0
    n=0
    for k in range (N):
        a=0
        for n in range(N):
            a=np.exp(((2j)*np.pi*k*n)/N)
            a=np.around(a,0) # this can be commented for checking with the built in fn
            w.append(a)
        w=np.array(w).reshape(N,N)
    return w

wc = (twiddlefactor(N))/N

h = x.dot(wc)

print("Impulse response h[n]:\n", h)
h_ift = np.fft.ift(x)
print("\nImpulse response h[n] (using np.fft.ift):\n", h_ift)

# Check if both results match
print("\nValidation (match with np.fft.ift):", np.allclose(h, h_ift))
```

Q3. Compute the autocorrelation of a rectangular pulse of width 10 samples. Plot the input and autocorrelation function and also comment on your results.

```
import numpy as np
import matplotlib.pyplot as plt

t=np.arange(-10,10,0.1)
T=10
x=np.zeros(len(t))
```

```

x[np.all([(t >= -T/2), (t <= T/2)], axis=0)] = 1

h=np.flip(x)
l = np.arange(-len(x) + 1, len(x))

N=len(x)+len(h)-1
y=[0]*N

for n in range(N):
    for k in range(len(h)):
        if 0<=n-k<len(x):
            y[n]+=x[n-k]*h[k]

plt.figure(figsize=(12, 6))

# Plot the rectangular pulse
plt.subplot(2, 1, 1)
plt.plot(t, x)
plt.title("Rectangular Pulse (Width 10 Samples)")
plt.xlabel("Time")
plt.ylabel("Amplitude")

# Plot the autocorrelation function
plt.subplot(2, 1, 2)
plt.stem(l * 0.1, y) # Scale lags by 0.1 to match time step
plt.title("Autocorrelation of Rectangular Pulse")
plt.xlabel("Lag")
plt.ylabel("Autocorrelation")

plt.tight_layout()
plt.show()

```

Q4. Linear convolution using circular convolution (both x and h is given)

```

import numpy as np

def dft(x):
    N = len(x)
    X = np.zeros(N, dtype=complex)
    for k in range(N):
        for n in range(N):
            X[k] += x[n] * np.exp(-1j * 2 * np.pi * k * n / N)

```

```

return X

def idft(X):
    N = len(X)
    x = np.zeros(N, dtype=complex)
    for n in range(N):
        for k in range(N):
            x[n] += X[k] * np.exp(1j * 2 * np.pi * n * k / N)
    return x / N

def linear_circular(x, h):
    L = len(x)
    M = len(h)
    N = L + M - 1

    # Zero-pad x and h to length N
    xp = np.array(x + [0] * (N - L))
    hp = np.array(h + [0] * (N - M))

    X = dft(xp)
    H = dft(hp)
    Y = X * H
    y = idft(Y)
    yr = np.real(y)
    return yr

# Input sequences
x = [int(i) for i in input("Enter x sequence: ").split(',')]
h = [int(i) for i in input("Enter h sequence: ").split(',')]

# Compute linear convolution using DFT
Y1 = linear_circular(x, h)
print("Answer:", Y1)
print("Check (using np.convolve):", np.convolve(x, h))

```

Q5. Find autocorrelation of $x(n)=0.25n$ for $0 \leq n \leq 5$. Plot input and output.

```

import numpy as np
import matplotlib.pyplot as plt

n = np.arange(0, 6)
x=0.25*n
t=np.arange(len(x))
x1=np.flip(x)

```



```

N=len(x)+len(x1)-1
y=[0]*N
l=np.arange(-len(x)+1, len(x))

for n in range(N):
    for k in range(len(x1)):
        if 0<=n-k<len(x):
            y[n]+=x[n-k]*x1[k]

y3=np.correlate(x, x, mode='full')

fig,a=plt.subplots(3,1, figsize=(10, 8))

a[0].stem(t, x)
a[0].set_title("Input Sequence x(n)")
a[0].set_xlabel("n")
a[0].set_ylabel("x(n)")

a[1].stem(l, y)
a[1].set_title("Manual Autocorrelation of x(n)")
a[1].set_xlabel("Lags")
a[1].set_ylabel("Autocorrelation")

a[2].stem(l, y3)
a[2].set_title("Autocorrelation of x(n) using np.correlate")
a[2].set_xlabel("Lags")
a[2].set_ylabel("Autocorrelation")

plt.tight_layout()
plt.show

```

Q. prove linearity and time shift property of dft

```
import numpy as np
```

```
x = np.array(input("Enter the elements of the input sequence :").split(",")).astype(int)
h = np.array(input("Enter the elements of the impulse response :").split(",")).astype(int)
```

```
hn = h
row = len(x)
column = len(x)+len(h)-1
```

```
if len(h)<column:
    z = np.zeros(column-len(h))
    h = np.concatenate((h,z))
```

```
#toeplitz matrix
toeplitz = h
for i in range (row-1):
    h = np.roll(h,1) #to circular shift elements of array
    toeplitz = np.concatenate((toeplitz,h))
```

```
toeplitz.shape = (row,column)
toeplitz = np.transpose(toeplitz)
```

```
print("Toeplitz Matrix :\n",toeplitz)
print("The output sequence (using toeplitz matrix) is :",np.dot(toeplitz,x).astype(int))
print("The output sequence (using numpy.convolve) is : ", np.convolve(x,hn))
```